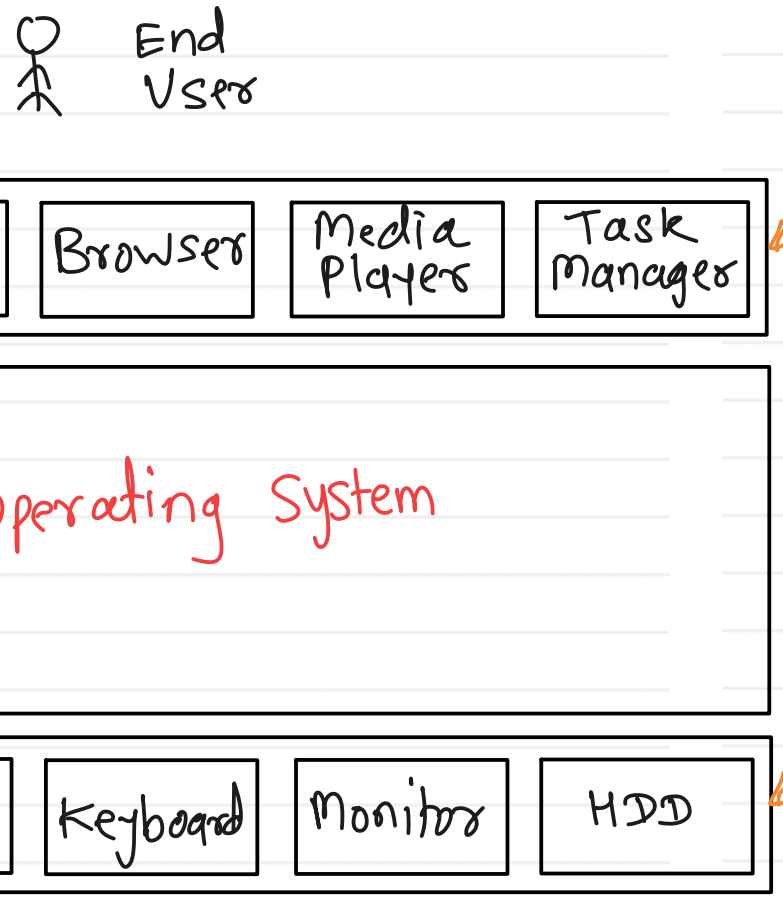
Sunbeam Institute of Information Technology
Pune and Karad

Embedded and RTOS (FreeRTOS) Training At RIT College, Islampur

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Operating system



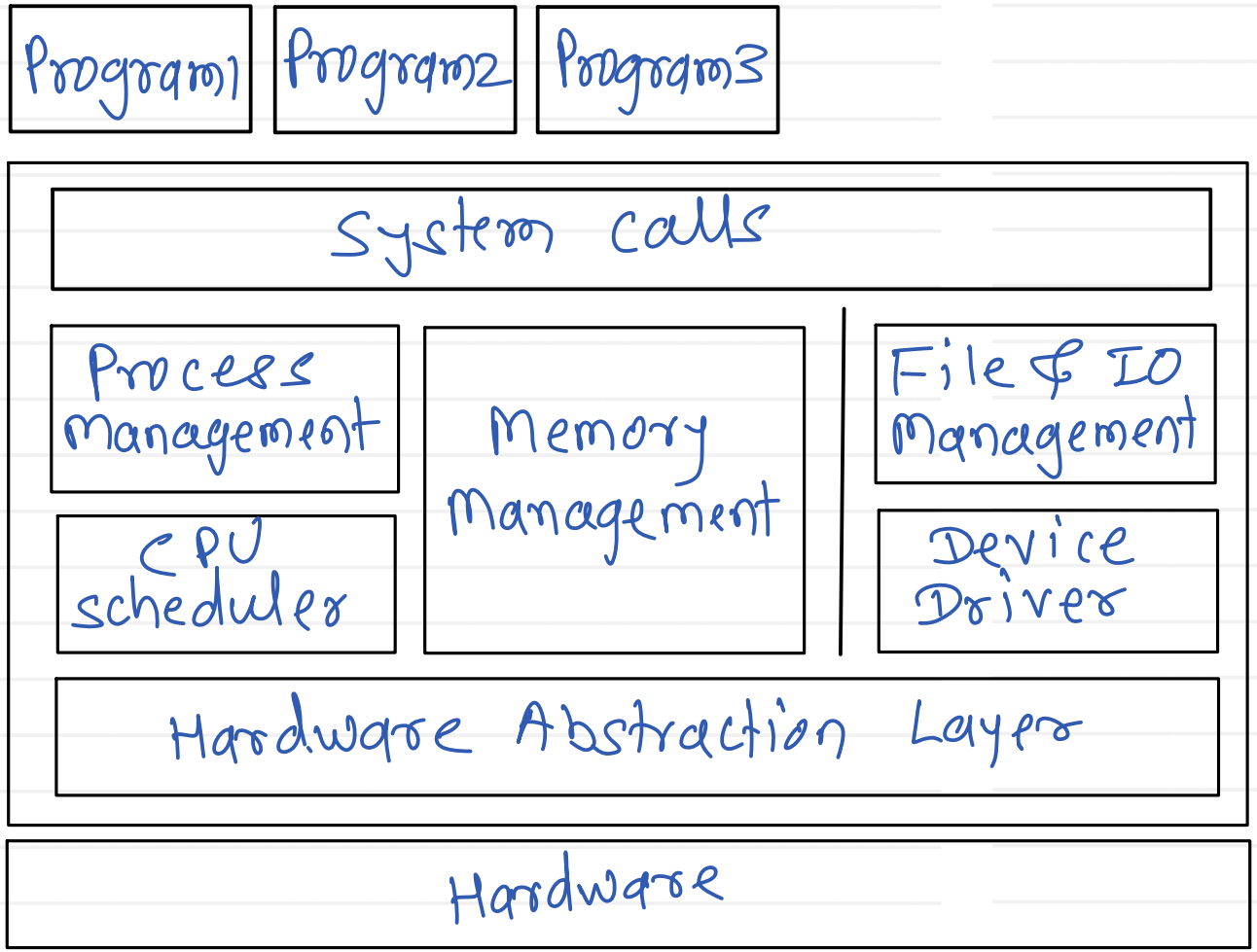
- interface bet^n H/W & end user.
- interface bet^n H/W & application s/w
- control program which controls execution of programs running in a system.
- resource allocator/manager which manages limited H/W resources in a system

CD/DVD/ISO = Core OS + System Utilities + Appl^n S/W

(Kernel)

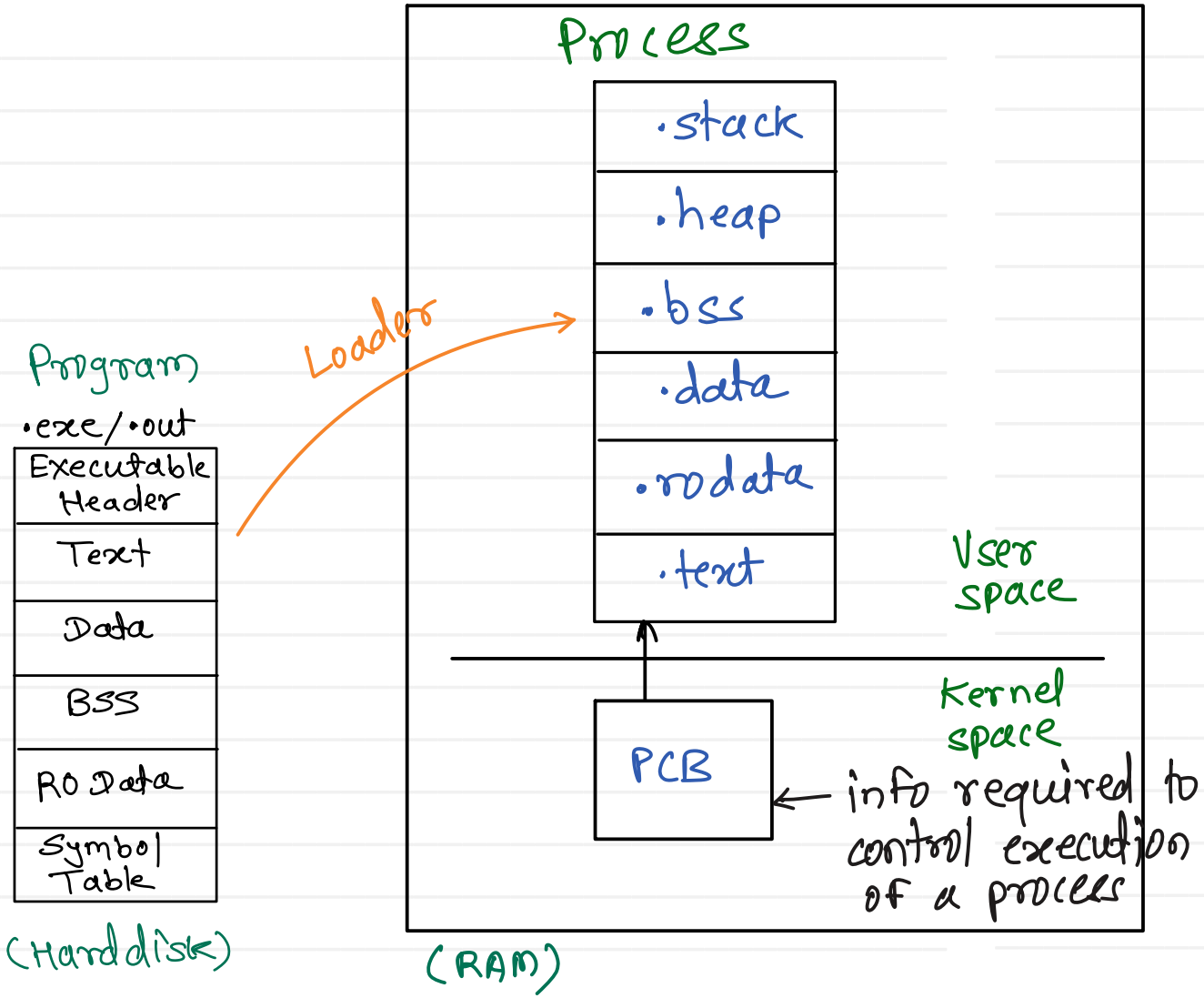
OS

Kernel functions



- User Interfacing
- Networking
- Security & Protection

Process



stack - FARs are stored
↳ return addr, local variables
Heap - dynamically allocated space

PCB - process descriptor
↓
struct task_struct
(sched.h)

- pid, ppid
- exit status
- Mem info
- Sched info
- IPC info
- Open file info
- execution context
- Kernel stack

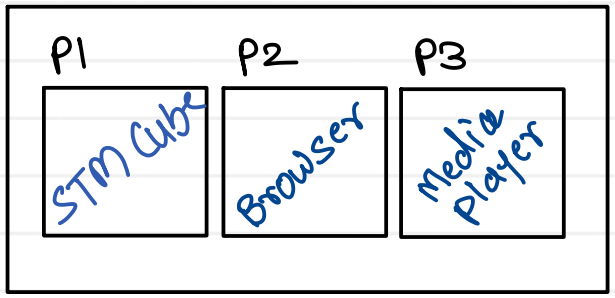
Time sharing systems / Multitasking

- CPU time is shared in all the processes of RAM

Response time < 1 sec

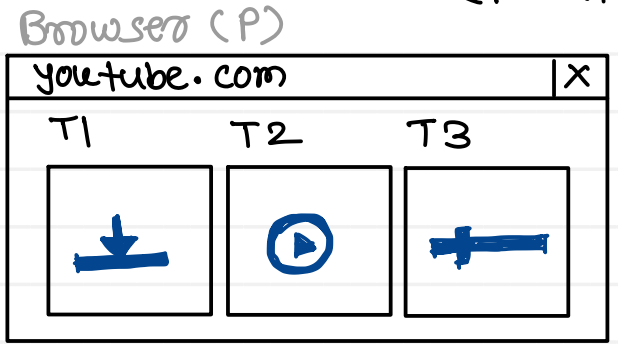
There are two types of multitasking.

i. Process based Multitasking



- system wide multitasking

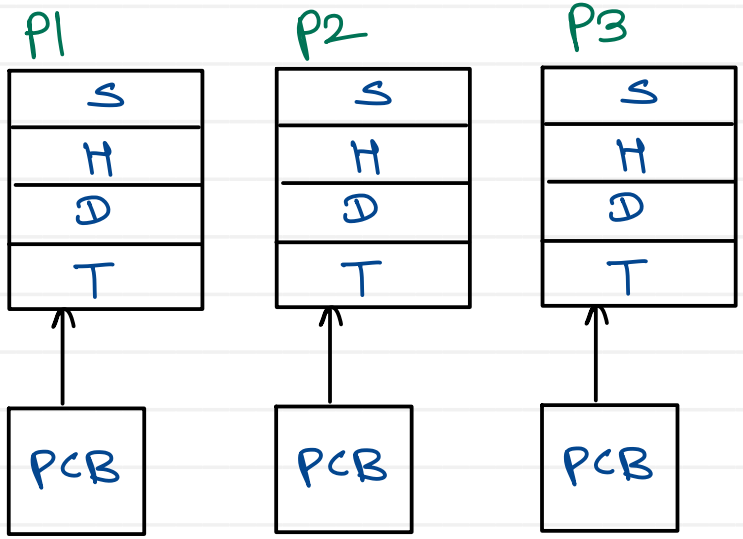
ii. Thread based Multitasking (Multi threading)



- Multitasking within process

Process Vs Thread

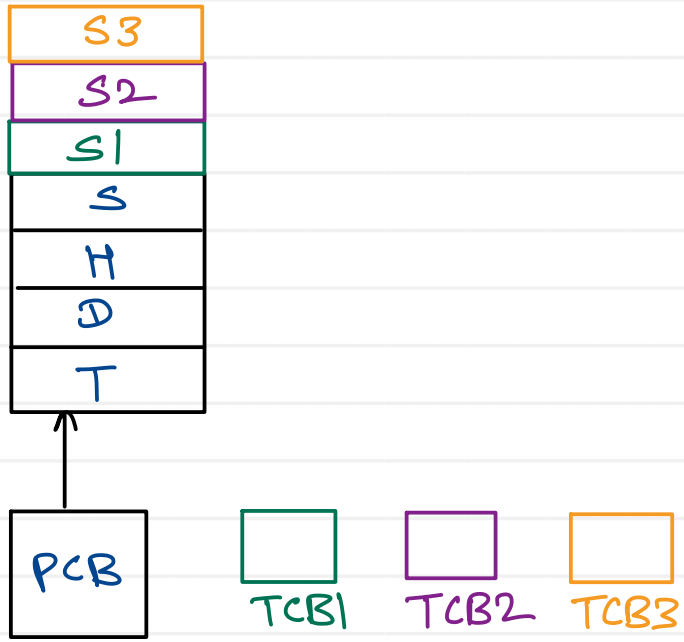
Processes are always created inside system



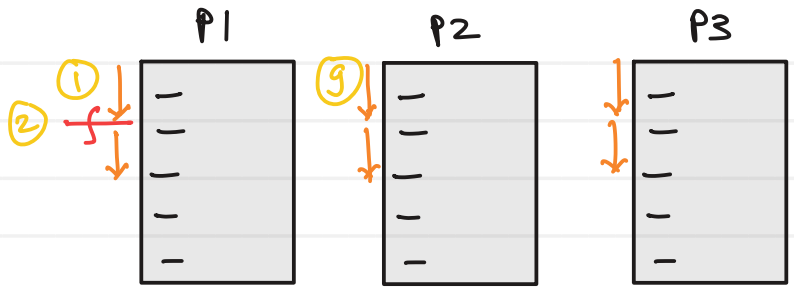
- Process is container of resources.
- Thread is alive entity which executes/run in container.
- every process has one default main thread.

Thread : light weight process

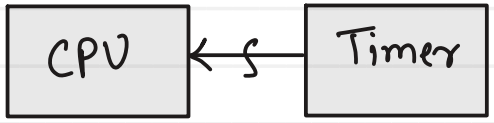
Threads are always created inside process



CPU Scheduling




```
interrupt_handler( ) {
    1. save execution of of current
       running process into its PCB
    2. Find address of ISR from IVT
       & call it
    3. pid = CPU_Scheduler( )
    4. CPU_Dispatcher(pid)
}
```




```
ISR()
{
    =
}
int CPU_Scheduler() {
    returns pid of selected process
}
void CPU_Dispatcher(pid) {
    load a given on a CPU
}
```

CPU Scheduling

- scheduler will be called on every timer interrupt

- frequency of timer interrupt is decided by one of the kernel configuration parameter

GPOS → CONFIG_HZ = 250 int/sec
RTOS → CONFIG_HZ = 1000 int/sec

- every timer interrupt is called as system tick. and it is counted by kernel.

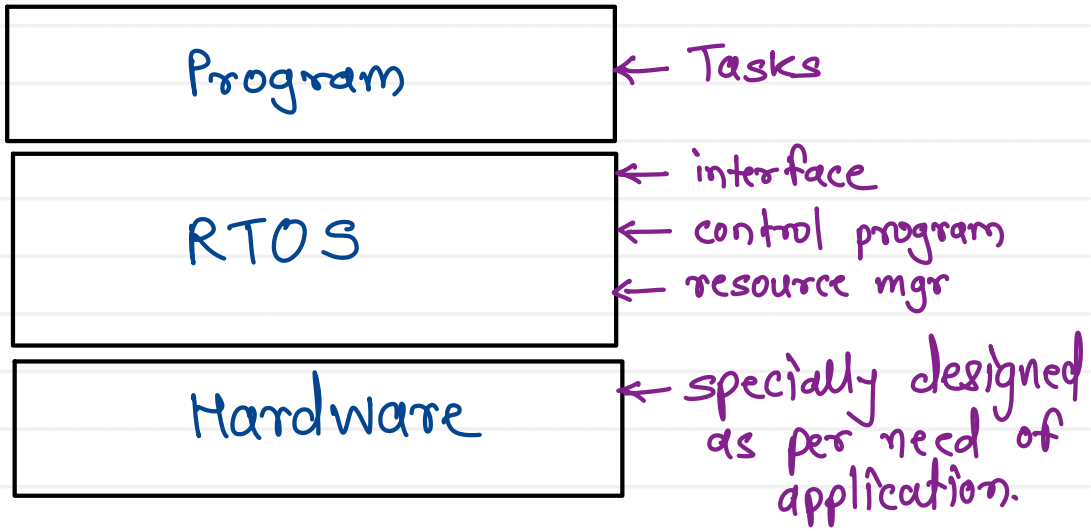
To select next process scheduler uses one of the algorithm:

1. FCFS
2. SJF
3. Priority
4. RR

- OS maintains a queue of ready processes (which are ready to execute on CPU) called as ready queue
- CPU scheduler always selects a process from ready queue

Process states : New, Ready, Running, Waiting, Terminated

RTOS



* result accuracy
* result should be calculated in deterministic time (minimal / fixed)
* less / deterministic interrupt latencies

Functionalities :

1) Task Management
2) Task scheduling
3) Memory Management
4) Timer Management
5) Interrupt / IO Management.
6) Hardware abstraction

RTOS examples :

FreeRTOS
uCOS
QNX
Zephyr
VxWorks
pSOS
RTLinux

Types of RTOS

- In RTOS to perform any task, we always have some deadline
- depending on strictness of deadline, we have 3 types of RTOS

1. Hard RTOS

- missing deadline will lead to some catastrophic effect

e.g. Air bag systems
pace makers
Air traffic controls
anti lock breaking systems
defense systems

2. Firm RTOS

- same like hard real time system, but occasionally missing deadline is acceptable
- it reduces product quality

e.g. reservation system,
online trading platform
production line automations

3. Soft RTOS

- time critical but missing deadline is tolerable & doesn't cause any catastrophic failure.

e.g. video conferency,
multimedia streaming
gaming

FreeRTOS

- Developed in collaboration with leading semiconductor/chip companies.
- More than 18 years of industry adoption and development.
- Market-leading Real-Time Operating System (RTOS) for:
  - Microcontrollers (MCUs)
  - Small Microprocessors (MPUs)
- Open-source software distributed under the MIT License.
- Lightweight and highly portable RTOS.
- Designed for embedded and resource-constrained systems.
- Supports deterministic real-time task scheduling.

- Consists of:
  - FreeRTOS Kernel
  - Growing set of middleware and libraries

- Widely used across multiple industry sectors:
  - Automotive
  - Industrial Automation
  - Consumer Electronics
  - Medical Devices
  - IoT & Smart Devices
  - Aerospace & Defense

- Large ecosystem and strong community support.
- Supported by numerous MCU vendors and development platforms.

Cortex-M Port of FreeRTOS

- The Cortex-M3 port includes all the standard FreeRTOS features:
  - Pre-emptive or co-operative operation
  - Very flexible task priority assignment
  - Queues
  - Binary semaphores
  - Counting semaphores
  - Recursive semaphores
  - Mutexes
  - Tick hook functions
  - Idle hook functions
  - Stack overflow checking
  - Trace hook macros
  - Optional commercial licensing and support

- FreeRTOS makes use of the Cortex-M3 SysTick, PendSV, and SVC interrupts.
- These interrupts are not available for use by the application.

- FreeRTOS has a very small footprint.
- A typical kernel build will consume approximately 6K bytes of Flash space and a few hundred bytes of RAM.
- Each task also requires RAM to be allocated for use as the task stack.

- FreeRTOS uses a modified GPL license. The modification is included to ensure:
  - FreeRTOS can be used in commercial applications.
  - FreeRTOS itself remains open source.
  - FreeRTOS users retain ownership of their intellectual property.

Task in FreeRTOS

- Tasks are implemented as C functions.
  - void ATaskFunction(void *pvParameters);

- Each task is a small program in its own right. It has an entry point, will normally run forever within an infinite loop, and will not exit.

```
void ATaskFunction( void *pvParameters ) {
        While(1)
        {
              ....
        }
}
```

- FreeRTOS tasks must not be allowed to return from their implementing function. If a task is no longer required, it should instead be explicitly deleted.

- A single task function definition can be used to create any number of tasks—each created task being a separate execution instance with its own stack.

Creating a Task in FreeRTOS

- xTaskCreate() API is used to create a task

```
portBASE_TYPE xTaskCreate(
        pdTASK_CODE pvTaskCode,
        const signed char * const pcName,
        unsigned short usStackDepth,
        void *pvParameters,
        unsigned portBASE_TYPE uxPriority,
        xTaskHandle *pxCreatedTask
);
```

- pvTaskCode - The pvTaskCode parameter is simply a pointer to the function that implements the task.
- const pcName - A descriptive name for the task.
- usStackDepth - The usStackDepth value tells the kernel how large to make the stack. The value specifies the number of words the stack can hold, not the number of bytes.
- pvParameters - The value assigned to pvParameters will be the value passed into the task.
- uxPriority - Defines the priority at which the task will execute. Priorities can be assigned from 0, which is the lowest priority, to (configMAX_PRIORITIES – 1), which is the highest priority.
- pxCreatedTask - used to pass out a handle to the task being created.